



Vietnam National University of HCMC
International University
School of Computer Science and Engineering



Object Oriented Programming

Method and a deeper look

Instructor: Le Thi Ngoc Hanh, Ph.D

ltnhanh@hcmiu.edu.vn

Semester – Spring 2026

Lecture Objectives

- ☾ Encapsulation concept
- ☾ Apply access modifiers correctly
- ☾ Design methods
- ☾ Distinguish instance vs static methods
- ☾ Understand parameter passing precisely
- ☾ UML Notion - Class diagrams

Design Problem

★ Problems

- No control over data integrity
- Violates business rules
- Anyone can corrupt object state
- No validation

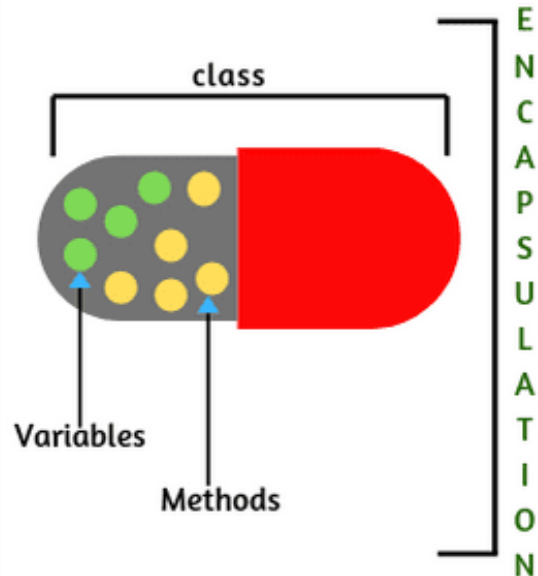
```
class BankAccount {  
    public double balance;  
}
```

```
BankAccount acc = new BankAccount();  
acc.balance = -1000;
```

Encapsulation

```
class  
{  
  
    data members  
    +  
    methods (behavior)  
  
}
```

E
N
C
A
P
S
U
L
A
T
I
O
N



data hiding!

Encapsulation

- ☾ Encapsulation is the mechanism of:
 - Bundling data (state) and behavior (methods)
 - Restricting direct access to internal data
 - Exposing only a controlled public interface
- ☾ Encapsulation ensures: Data integrity, Controlled interaction, Protection of invariants, Clear object responsibility

Encapsulation – Important Terms

- ★ The **internal state** of an object is the collection of its attribute values stored in memory that represent its current condition.

```
class BankAccount {  
    private double balance;  
}
```

- ★ **Public interface** of a class is the set of public methods that allow external code to interact with the object.

```
public void deposit(double amount)
```

Encapsulation – Important Terms

- ★ **Information hiding** is the principle of restricting access to implementation details and exposing only what is necessary.

```
private double balance;  
// --> account.balance = -5000; // illegal
```

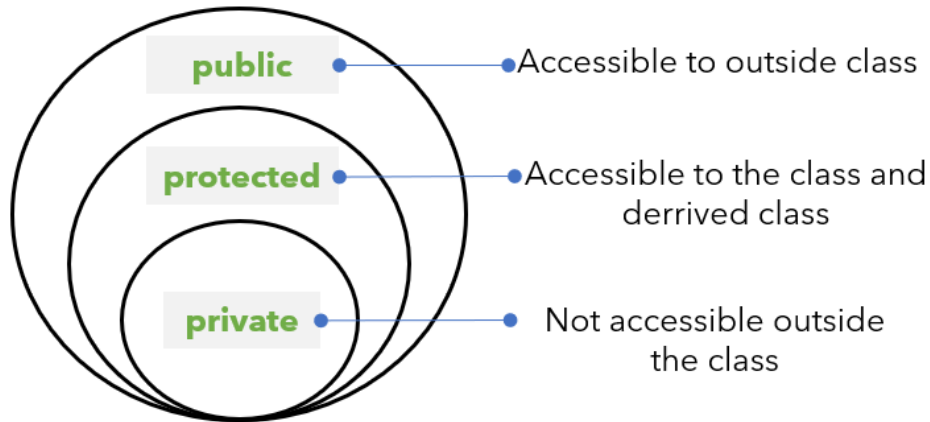
- ★ An **invariant** is a condition that must always remain true for an object to be in a valid state.

```
public void withdraw(double amount) {  
    if(amount > 0 && amount <= balance) {  
        balance -= amount;  
    }  
}
```

Access Modifiers in Java

Access modifiers define:

- Who can access a class member (field or method)
- The visibility scope of attributes and behaviors
- The level of encapsulation enforced
- The boundary of responsibility



Encapsulation – Sample

```
class BankAccount {  
    private double balance;  
  
    protected String accountType;  
  
    public String ownerName;  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
}
```

Getter & Setter

- ☾ A getter is a public method that provides controlled read access to a private field.
 - A getter allows external code to retrieve the value of an internal attribute without exposing the field directly.
- ☾ A setter is a public method that allows controlled modification of a private field.
 - A setter ensures that any change to internal state passes through validation logic.

Syntax

```
public Type getFieldName() {  
    return fieldName;  
}
```

```
public void setFieldName(Type fieldName) {  
    this.fieldName = fieldName;  
}
```

Example

```
private double balance;
```

```
public double getBalance() {  
    return balance;  
}
```

```
public void setBalance(double balance) {  
    this.balance = balance;  
}
```

Method

☾ A method is:

- A behavior defined inside a class
- An action that an object can perform
- A method is a behavior defined inside a class that allows an object to perform an action or provide information.
- Example:

Class

Behaviors



person

work ()
sleep()



Bank_Account

check_balance ()
withdraw()

Object Structure Refined

★ An Object consists of:

- Internal State
- Behavior (methods operating on state)

★ Encapsulation principle:

- State is hidden
- Behavior is exposed

★ Well-designed object:

- Protects its internal state
- Controls how state changes occur
- Maintains logical consistency

class

{

data members

+

methods (behavior)

}

ENCAPSULATION

Method

```
[access] returnType methodName(parameterList) {  
    // implementation  
}
```

```
public double calculateInterest(double rate) {  
    return balance * rate;  
}
```

Parameter Passing in Java

Java pass-by-value ONLY!

★ Primitive Type

- Value copied
- Original unchanged

★ Object Reference

- Reference value copied
- Object not copied
- Object state can change
- Reference variable cannot be reassigned externally

```
public class PrimitiveExample {  
  
    public static void changeValue(int number) {  
        number = 100;  
        System.out.println("Inside method: " + number);  
    }  
  
    public static void main(String[] args) {  
        int x = 10;  
  
        changeValue(x);  
  
        System.out.println("After method call: " + x);  
    }  
}
```

```
class Student {  
    String name;  
    public Student(String name) {  
        this.name = name;  
    }  
}  
  
public class ObjectExample {  
  
    public static void changeStudent(Student s) {  
        s.name = "Lan";    // modify object state  
        System.out.println("Inside method: " + s.name);  
    }  
  
    public static void main(String[] args) {  
  
        Student st = new Student("John");  
        changeStudent(st);  
        System.out.println("After method call: " + st.name);  
    }  
}
```

```
public class Example {  
  
    public static void reassignStudent(Student s) {  
        s = new Student("David");  
        System.out.println("Inside method: " + s.name);  
    }  
  
    public static void main(String[] args) {  
  
        Student st = new Student("John");  
  
        reassignStudent(st);  
  
        System.out.println("After method call: " + st.name);  
    }  
}
```

Return Types

★ A return type defines:

- The output interface of the method
- The data contract between method and caller
- What the caller is allowed to receive and use

★ Three categories:

- Void
- Primitive return: returns data value
- Object return: returns reference to object

Return Types - Sample

```
public void deposit(double amount) {  
    balance += amount;  
}
```

```
public double getBalance() {  
    return balance;  
}
```

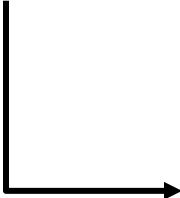
```
public Student createStudent(String name) {  
    return new Student(name);  
}
```

Static vs Instance Methods

★ Instance Method

- Requires an object instance to be invoked
- Operates on the internal state of that specific object
- Has access to instance variables and this reference
- Represents behavior belonging to an object
- An instance method can only be called through an object reference

```
class BankAccount {  
    private double balance;  
    public void deposit(double amount) {  
        balance += amount;  
    }  
    public double getBalance() {  
        return balance;  
    }  
}
```



```
public class Test {  
    public static void main(String[] args) {  
        BankAccount acc = new BankAccount();  
        acc.deposit(1000);  
        System.out.println(acc.getBalance());  
    }  
}
```

Static vs Instance Methods

★ Static Method

- Belongs to the class, not to any specific object
- Can be invoked without creating an object
- Cannot directly access instance variables
- Does not have access to this reference

Static Method - Sample

```
class MathUtil {  
    public static int add(int a, int b) {  
        return a + b;  
    }  
}  
public class Test {  
    public static void main(String[] args) {  
        int result = MathUtil.add(3, 5);  
        System.out.println(result);  
    }  
}
```

Method Overloading

- ☾ Method overloading occurs when multiple methods:
 - Have the same method name
 - Exist within the same class
 - Differ in their parameter list (number, type, or order of parameters)
 - Are resolved at compile time
- ☾ Method overloading allows a class to define multiple versions of logically related behavior under a single method name.

Method Overloading

★ Valid overloading: the parameter list must differ in at least one of the following ways:

- Different number of parameters
- Different parameter types
- Different parameter order

```
public void deposit(double amount)
public void deposit(double amount, String note)
public void deposit(int amount)
```

Method Overloading

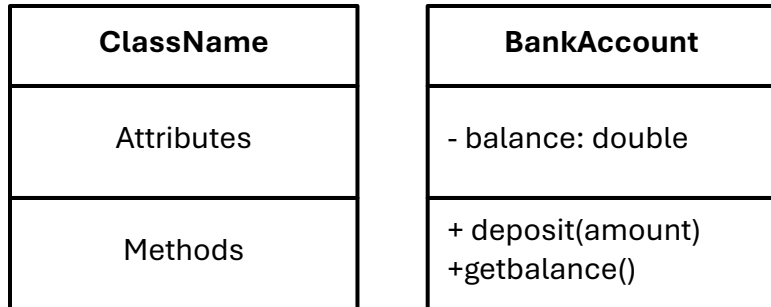
☾ Method overloading improves:

- Code readability
- Logical grouping of related behaviors
- API usability
- Flexibility for method calls

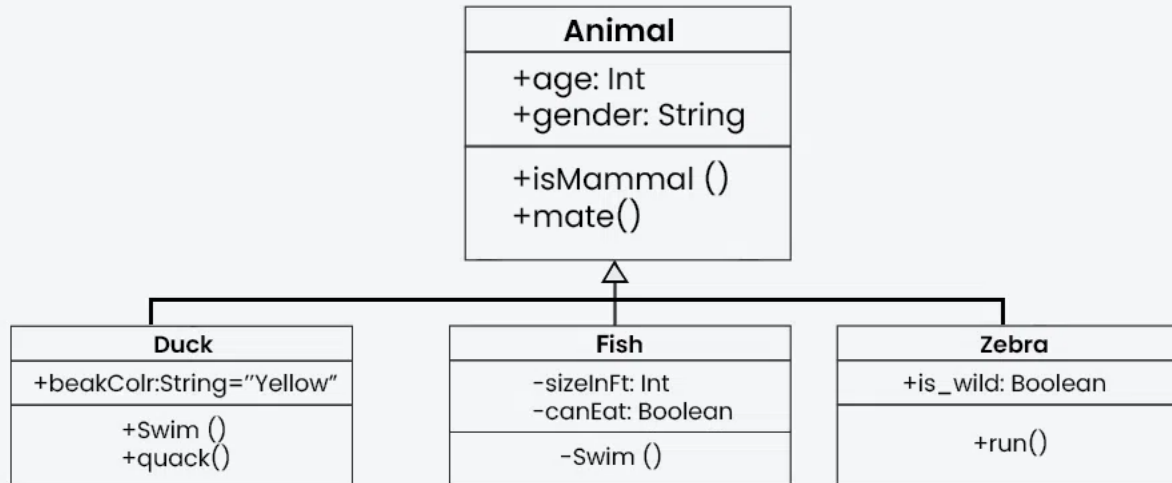
UML Notion - Class Diagram

★ UML Class Diagram represents:

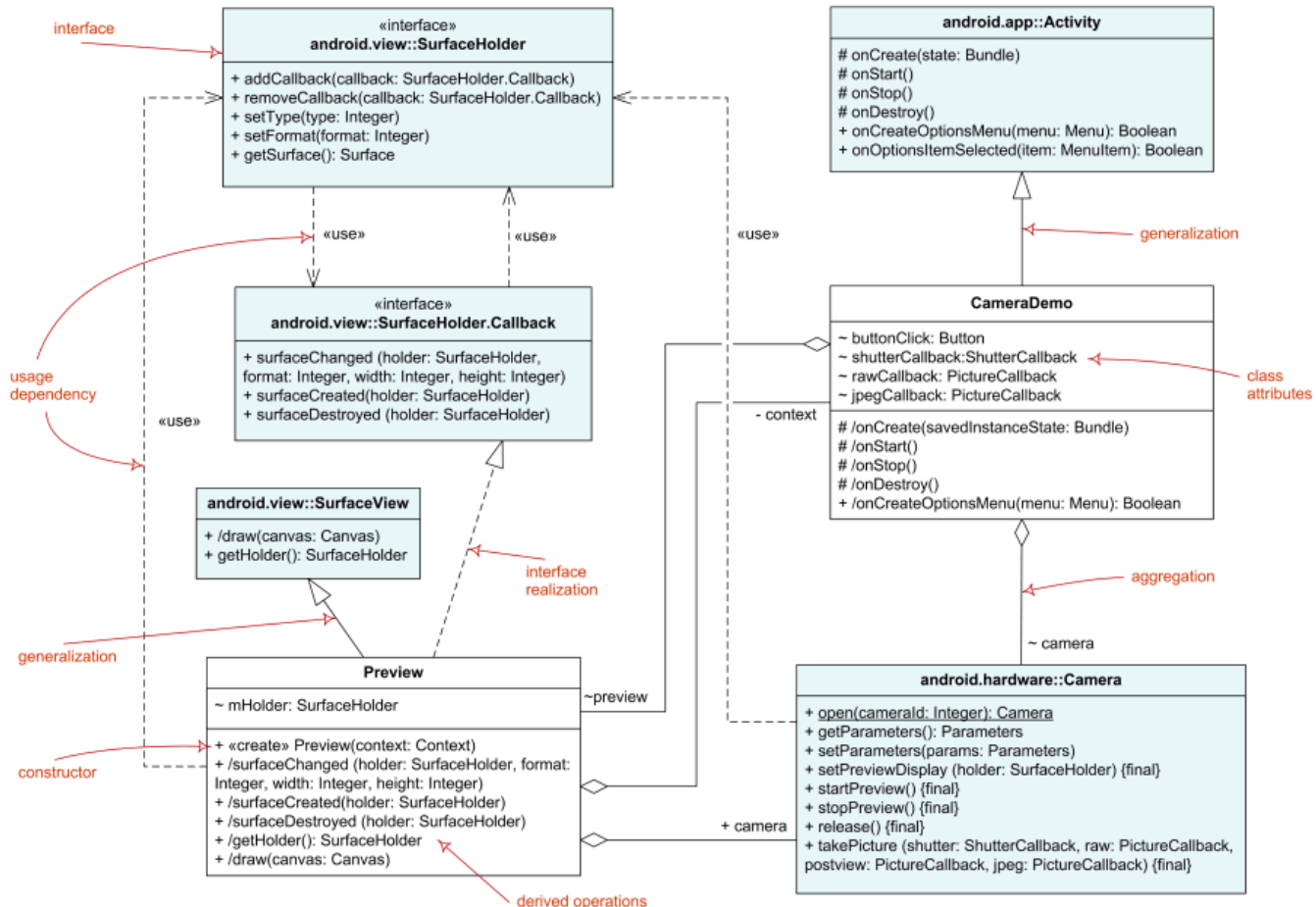
- Class structure
- Attributes
- Methods
- Visibility



UML Notion - Class Diagram



Source: <https://www.geeksforgeeks.org>



UML ↔ Java Mapping

BankAccount
- balance: double
+ deposit(amount) + getbalance()

```
class BankAccount {  
    private double balance;  
  
    public void deposit(double amount) { ... }  
    public double getBalance() { ... }  
}
```